sercon

User Manual

Version 0.7.0

2026-05-26

Repository · https://github.com/codedeviate/sercon

sercon manual

Long-form reference for the `pkg/scriptengine` library, the `sercon` CLI, the built-in script API, and the JavaScript runtime surface that `scriptengine` exposes via goja and goja_nodejs. Examples are runnable — save snippets as `.ts` files and execute with `sercon file.ts` unless otherwise noted.

Table of contents

---

1. Overview

`sercon` is a Go program that runs TypeScript files. It's built from two parts:

- `pkg/scriptengine` — a library you embed in your own Go code. You register Go-callable bindings, then call `Run` / `RunFile` to execute a `.ts` script that talks to them.
- `cmd/sercon` — a thin CLI built on the library, useful for ad-hoc test scripts and as a working example of every binding kind.

The runtime is pure Go: goja is the JS engine, esbuild (used as a Go library) is the TS→JS transpiler, and goja_nodejs provides Promises, `setTimeout`, `console`, and CommonJS `require`. No cgo, no Node.

2. Quickstart

Install:

```
go install github.com/codedeviate/sercon/cmd/sercon@latest
```

Run one of the bundled examples:

```
sercon examples/scripts/smoke.ts examples/scripts/async.ts
```

…or run them all via `make demo`. `examples/scripts/` ships a runnable `.ts` (or `.tsx`) file per feature area — `hash.ts`, `strings.ts`, `path-and-time.ts`, `default-export.ts`, `tsx-demo.ts`, and the original `smoke.ts` / `async.ts` / `hang.ts` (`hang.ts` is the timeout demo and intentionally exits non-zero).

Generate a declaration file for editor autocomplete:

```
sercon --emit-dts api.d.ts
```

Embed in your own Go program:

```
eng := scriptengine.New(scriptengine.Options{
    Timeout:    5 * time.Second,
    ScriptRoot: "./scripts",
})
eng.Register("upper", strings.ToUpper)
val, err := eng.RunFile(ctx, "scripts/main.ts")
```

3. Library API — `pkg/scriptengine`

`Engine`, `Options`, `New`

```
type Options struct {
    Timeout        time.Duration // wall-clock limit per Run; 0 disables
    ScriptRoot     string        // base for require/import resolution
    DisableConsole bool          // turn off the goja_nodejs console module
    Verbose        io.Writer     // diagnostic traces, prefixed "[sercon] "
    ModuleLoader   func(candidatePath string) (source string, found bool,
err error)
}

func New(opts Options) *Engine
```

`ModuleLoader`, when non-nil, is consulted for every require/import candidate path **before** the filesystem — the hook for embedders that want to serve modules from somewhere other than disk (an in-memory FS, a network source, an embedded bundle). goja probes several candidates per specifier (`./x`, `./x.ts`, `./x/index.ts`, …); the engine also tries the bare path plus the usual extension fallbacks (`.ts` / `.tsx` / `.js` / `.mjs` / `.cjs` / `.json`) against the loader, so a loader can match on a plain suffix. Return `(source, true, nil)` to serve the module (transpiled when the path ends in `.ts` / `.tsx`, exactly like a disk read); `("", false, nil)` to fall through to the filesystem; `("", false, err)` to abort resolution.

```
eng := scriptengine.New(scriptengine.Options{
    ModuleLoader: func(path string) (string, bool, error) {
        if strings.HasSuffix(path, "greeting.ts") {
            return `export const hi = (n: string) => "hi " + n;`, true, nil
        }
        return "", false, nil // fall through to disk
    },
})
```

`Engine` is the host of all registered bindings. Construct it once, then call `Run`/`RunFile` as many times as needed — each call gets a *fresh* `*goja.Runtime` so script state never leaks between invocations.

`ScriptRoot` defaults to the working directory at `New` time if left empty. Both `Timeout` and `ScriptRoot` can be overridden on a per-Run basis only by constructing a fresh `Engine`; see Out-of-scope for the per-Run override on the roadmap.

Registering bindings

Function	Use when…
`Register(name, value)`	The binding is a pure function, struct, or primitive that doesn't need access to the runtime.
`RegisterNamespace(name, members map[string]any)`	You want to expose `name.x`, `name.y` without defining a Go struct.
`RegisterConstructor(name, ctor)`	The binding is a Go function whose return value should be treated as a class in the emitted `.d.ts`. (Runtime semantics today are identical to `Register`; the d.ts emitter is the only differentiator.)
`RegisterFactory(name, factory func(vm, loop) any)`	The binding needs the per-Run `*goja.Runtime` or `*eventloop.EventLoop` in scope (typical for Promise-returning I/O).

Function	Use when…
`RegisterNamespaceFactory(name, factory)`	Same, but the factory returns the full member map.

Example — synchronous function binding:

```
eng.Register("greet", func(name string) string { return "hi " + name })
```

Example — namespace with a sync member:

```
eng.RegisterNamespace("math2", map[string]any{
    "square": func(n float64) float64 { return n * n },
    "pi":     3.14159,
})
```

Example — async (Promise-returning) binding via `PromisifyAsync`:

```
eng.RegisterFactory("httpGet", func(vm *goja.Runtime, loop
*eventloop.EventLoop) any {
    return scriptengine.PromisifyAsync(vm, loop,
        func(ctx context.Context, call goja.FunctionCall) (map[string]any,
error) {
            url := call.Argument(0).String()
            resp, err := http.Get(url)
            if err != nil {
                return nil, err
            }
            defer resp.Body.Close()
            body, _ := io.ReadAll(resp.Body)
            return map[string]any{"status": resp.StatusCode, "body":
string(body)}, nil
        })
})
```

`Run`, `RunFile`

```
func (e *Engine) Run(ctx context.Context, name, source string, opts
...RunOption) (goja.Value, error)
func (e *Engine) RunFile(ctx context.Context, path string, opts
...RunOption) (goja.Value, error)
```

`Run` executes `source` as an entry-script TS. `name` is used in stack traces. The returned value is currently always `undefined` — top-level expression capture is on the backlog.

Both methods respect `Options.Timeout` *and* `ctx` cancellation, whichever fires first; the resulting error is either `ErrScriptTimeout`, `ctx.Err()`, or the underlying JS exception.

Per-Run options

```
type RunOption func(*runConfig)

func WithScriptRoot(dir string) RunOption
```

Reuse a single Engine across many scripts that each live in their own directory by passing `WithScriptRoot(dir)` to `Run` / `RunFile`. The override applies only to this call; `Options.ScriptRoot` is used for every other Run.

```
_, _ = eng.Run(ctx, "main.ts", source,
scriptengine.WithScriptRoot("/path/to/run42"))
```

```
func WithArgs(args []string) RunOption
```

Set the per-script argument vector. The script reads it as `Sercon.argv`, a global with Node/Bun layout: `argv[0]` is `Options.ProgramName` (defaults to `filepath.Base(os.Args[0])`; the CLI sets it to `"sercon"`), `argv[1]` is the running script's path, and the `WithArgs` values follow from index 2. The global is always present — with no args it is just `[programName, scriptPath]`.

```
_, _ = eng.Run(ctx, "main.ts", source, scriptengine.WithArgs([]string{"--
port", "8080"}))
// inside the script: Sercon.argv === [ProgramName, "/abs/main.ts", "--
port", "8080"]
```

`Reset`

```
func (e *Engine) Reset()
```

Clears every registered binding. Useful when a long-lived Engine is reused across unrelated batches of scripts that each want a clean global namespace. **Not safe to call concurrently with Run / RunFile.**

`WriteTypes`

```
func (e *Engine) WriteTypes(w io.Writer) error
```

Emits a `.d.ts` describing the registered surface. See section 13. Type generation for what the mapping looks like.

`SetDocs` and `SetMemberDocs`

```
func (e *Engine) SetDocs(path, doc string)
func (e *Engine) SetMemberDocs(namespace string, docs map[string]string)
```

Attach JSDoc strings to registered bindings so the emitted `.d.ts` grows readable editor hover. `SetDocs` takes a dotted path — either a bare top-level name (`"greet"`, `"http"`) or `"namespace.member"` for namespace members (`"http.get"`, `"exec.shell"`); `SetMemberDocs` is the convenience for documenting many members of a namespace at once (the namespace itself can still be documented separately via `SetDocs`).

Multi-line docs are written as `\n`-separated strings; the emitter expands them into a standard `* `-prefixed JSDoc block. Single-line docs collapse to `/** … */`. Calling `SetDocs` with an empty string removes any previous doc for that path. Bindings without a doc entry get no JSDoc block at all — the emitter doesn't insert empty `/** */` placeholders.

`SetDocs` may be called before or after the matching `Register…` call; docs are looked up by name at emit time. Like the registration methods, `SetDocs` must not race with a `Run` / `WriteTypes` / `Reset` on the same engine.

```
eng.Register("greet", func(name string) string { return "hi " + name })
eng.SetDocs("greet", "Greet someone by name.")

eng.RegisterNamespace("math2", map[string]any{
    "pi":      3.14159,
    "squared": func(n float64) float64 { return n * n },
})
eng.SetDocs("math2", "Tiny math helpers.")
eng.SetMemberDocs("math2", map[string]string{
    "pi":      "Circumference / diameter ratio.",
    "squared": "Multiply a number by itself.",
})
```

`PromisifyAsync[T]` and `AsyncBinding`

```
type AsyncBinding struct {
    Func         func(goja.FunctionCall) goja.Value
    TSReturnType string
}

func PromisifyAsync[T any](vm *goja.Runtime, loop *eventloop.EventLoop,
    work func(ctx context.Context, call goja.FunctionCall) (T, error),
) AsyncBinding
```

`PromisifyAsync` turns blocking Go work into a JS Promise. It launches the work in a goroutine, parks a `SetTimeout` sentinel so `loop.Run` doesn't return early, and schedules the resolve/reject back onto the event loop. **Required** for any Promise-returning binding — `RunOnLoop` alone is not counted as a live job by the event loop.

`PromisifyAsync` returns an `AsyncBinding` carrier rather than the bare callback. The engine unwraps it to `.Func` at `vm.Set` time so goja's special-case detection of `func(goja.FunctionCall) goja.Value` host-callbacks still fires; the `.d.ts` emitter reads `.TSReturnType` to emit `Promise<T>` (mapped from the generic `T` via reflect at construction time) instead of the previous `unknown`. Hosts assigning the result into a `map[string]any` for `RegisterNamespace` / `RegisterNamespaceFactory` get the unwrap recursively too — no manual work required.

Version

```
import "github.com/codedeviate/sercon/pkg/scriptengine"

fmt.Println(scriptengine.Version) // "0.1.0"
```

Bumped in lockstep with the git tag.

4. CLI — `sercon`

```
sercon [flags] <script.ts> [script.ts ...]
sercon [flags] -                # read entry script from stdin
sercon --examples | --help | --version
```

Flag	Purpose
`-timeout DURATION`	Wall-clock limit per script (default `10s`; `0` disables).
`-root DIR`	Override the script root for `require`/`import` resolution.
`-emit-dts PATH`	Write the example bindings' `.d.ts` to `PATH` and exit.
`-v`	Verbose: trace the rewritten entry-script JS and each module resolution to stderr; also print duration on failure.
`-h`, `--help`	In-depth colorized help.
`--examples`	In-depth colorized walkthrough of every feature.
`--version`	Print the engine version (plus goja/esbuild build-info versions).
`--watch`	Re-run on file change under the script root. Debounced 150 ms. Ctrl-C exits.

Each positional argument is either a path to a `.ts` / `.tsx` file or `-` to read an entry script from standard input:

```
echo 'api.runtime.log(1 + 2);' | sercon -
sercon prelude.ts -                # prelude then stdin
```

Passing arguments: `--` and `Sercon.argv`

Everything after a standalone `--` is the script argument vector, exposed to every script as the `Sercon` runtime global:

```
sercon run.ts -- --port 8080 hello
```

`Sercon.argv` uses the Node/Bun layout `[programName, scriptPath, ...userArgs]` — `argv[0]` is `"sercon"`, `argv[1]` is the path of the script currently executing, and the post-`--` arguments start at index 2 (so `Sercon.argv.slice(2)` is just your args). All scripts in one invocation share the same `--` tail, but each sees its own path at `argv[1]`. The global is always present; with no `--` it is just `[programName, scriptPath]`.

```
const args = Sercon.argv.slice(2);   // ["--port", "8080", "hello"]
```

Exit codes are distinct per failure type so shells can react sensibly. When several scripts run, the highest applicable code wins:

Code	Meaning
`0`	every script passed.
`1`	CLI usage error (unknown flag, missing script argument, …).
`2`	at least one script failed to transpile — never ran.
`3`	at least one script timed out (`-timeout`) or was context-cancelled.
`4`	at least one script ran and threw a JS exception.

`-v` writes lines prefixed with `[sercon] ` to stderr. The traces include the full rewritten entry-script JS (the form goja actually runs, after the ESM→CJS rewrite + async IIFE wrapper) and every module-resolution event, so debugging an unexpected resolve target or a mis-rewritten import is straightforward.

The CLI registers a single `api` namespace; see the next section.

`--watch`: re-run on file change

`sercon --watch <script.ts>` runs the script once and then blocks, re-running it every time a watched file changes under the script root. Useful for iterating on a script body or on the binding surface during development.

```
sercon --watch examples/scripts/smoke.ts
```

What's watched:

- The script root (resolved the same way as the one-shot mode — `-root DIR` if set, else `dirname` of the first script argument).
- Recursively. Symlinks aren't followed.
- New directories that appear during the session are picked up automatically.

What's filtered:

- **File extensions**: `.ts` / `.tsx` / `.js` / `.jsx` / `.json` trigger a re-run; `.d.ts` files match via suffix too. Anything else (Markdown, images, editor swap files, build artefacts) is ignored.
- **Directories**: hidden directories (`.git` , `.vscode` , anything starting with `.`) and `node_modules` are excluded — both because they generate floods of irrelevant events and because recursing into them inflates the watcher's directory count for no gain.

Re-runs are **debounced 150 ms**. Editors typically fire several events per save (write → rename → chmod); collapsing them into a single re-run keeps the loop responsive without thrashing. The debounce window resets on every event, so a rapid burst of saves becomes one re-run after the burst settles.

Each run is delimited by a line like `--- sercon re-run @ HH:MM:SS ---` so the output is visually distinct from the previous run.

`Ctrl-C` (SIGINT) or `SIGTERM` exit cleanly. Per-script failures inside a watch session log as `FAIL` but don't terminate the loop — a syntax error you're trying to fix shouldn't kick you out of watch mode.

The watcher reuses the same `Engine` across runs. Each `Run` already gets a fresh `*goja.Runtime` , so re-running is just re-invoking `runOne` on the existing engine — there's no per-iteration setup cost beyond the transpile + module-resolution work.

Module-graph invalidation. Watch mode tracks each entry script's import graph (its own file plus every module it resolves, captured via `Engine.SetResolveHook`) during the run. On a file change it re-runs **only the entries whose graph includes the changed file** — editing a helper that just one of three entry scripts imports re-runs that one entry, not all three. An entry's own file counts (editing it re-runs it); stdin entries and any entry whose graph isn't known yet re-run unconditionally (conservative). Before each re-run the module cache is busted (`Engine.ResetModuleCache`) so an edited import's new source actually takes effect — the registry otherwise caches compiled bytecode across runs.

5. Built-in script `api`

Migrating from 0.7

`api.*` was reorganised in 0.8.0 from 32 flat groups into 9 category buckets. Every script written for 0.7 or earlier needs path updates. The mapping:

Old path	New path
`api.log`	`api.runtime.log`
`api.assert.equal`	`api.runtime.assert.equal`
`api.assert.ok`	`api.runtime.assert.ok`
`api.time.*`	`api.runtime.time.*`
`api.env.get`	`api.runtime.env.get`
`api.hash.*`	`api.crypto.hash.*`
`api.jwt.*`	`api.crypto.jwt.*`
`api.encrypt.*`	`api.crypto.encrypt.*`
`api.str.*`	`api.text.str.*`
`api.preg.*`	`api.text.preg.*`
`api.preg2.*`	`api.text.preg2.*`
`api.text.detect`	`api.text.charset.detect`
`api.text.encode`	`api.text.charset.encode`
`api.text.decode`	`api.text.charset.decode`
`api.jq.*`	`api.text.jq.*`
`api.diff.compare`	`api.text.diff.compare`
`api.compression.*`	`api.format.compression.*`
`api.barcode.*`	`api.format.barcode.*`
`api.checkdigit.*`	`api.format.checkdigit.*`
`api.path.*`	`api.fs.path.*`
`api.archive.*`	`api.fs.archive.*`
`api.http.*`	`api.net.http.*`
`api.net.tcp`	`api.net.probe.tcp`
`api.net.dns`	`api.net.probe.dns`
`api.net.tls`	`api.net.probe.tls`
`api.net.ntp`	`api.net.probe.ntp`
`api.net.whois`	`api.net.probe.whois`

Old path	New path
`api.net.ping`	`api.net.probe.ping`
`api.net.smtp`	`api.net.probe.smtp`
`api.net.wss`	`api.net.probe.wss`
`api.netstatus.*`	`api.net.netstatus.*`
`api.email.*`	`api.net.email.*`
`api.browser.*`	`api.net.browser.*`
`api.sqlite.*`	`api.db.sqlite.*`
`api.redis.*`	`api.db.redis.*`
`api.memcached.*`	`api.db.memcached.*`
`api.ldap.*`	`api.db.ldap.*`
`api.dict.*`	`api.db.dict.*`
`api.exec.shell`	`api.tools.exec.shell`
`api.exec.http`	`api.tools.exec.http`
`api.git.*`	`api.tools.git.*`
`api.gh.*`	`api.tools.gh.*`
`api.ai.*`	`api.tools.ai.*`
`api.tui.layout`	`api.ui.tui.layout`
`api.tui.pane`	`api.ui.tui.pane`

Every leaf function name stays the same; only the addressing changes. A mechanical sed pass over your scripts handles it — see the `CHANGELOG.md` 0.8.0 "Changed" entry for the canonical sed command.

The CLI exposes the following globals to every script:

```
declare const api: {
  log(...args: unknown[]): void;

  assert: {
    equal(actual: unknown, expected: unknown, msg?: string): void;
    ok(cond: unknown, msg?: string): void;
  };

  http: {
    get(url: string): Promise<{ status: number; body: string }>;
    post(url: string, body?: string): Promise<{ status: number; body:
string }>;
    request(
      method: string,
      url: string,
      opts?: {
        headers?: Record<string, string>;
        body?: string;
        timeout?: number;        // ms; default 30000
        retry?: number;          // re-attempts on transport error / 5xx;
default 0
        follow?: boolean;        // follow redirects; default true
        username?: string;       // basic auth
        password?: string;
      },
    ): Promise<{
      status: number;
      ok: boolean;               // status in [200, 400)
      headers: Record<string, string>;  // lower-cased names
      body: string;
      url: string;               // final URL after redirects
    }>;
  };

  time: {
    nowMs(): number;
    sleep(ms: number): Promise<void>;
    format(unixMs: number, fmt: string, tz?: string): string;
  };

  env: {
    get(name: string): string | undefined;
  };

  str: {
    trim(s: string, mask?: string): string;
    ltrim(s: string, mask?: string): string;
    rtrim(s: string, mask?: string): string;
```

```
    reverse(s: string): string;
    stripHtml(s: string): string;
    nl2br(s: string, xhtml?: boolean): string;
    br2nl(s: string): string;
    base64Encode(s: string): string;
    base64Decode(s: string): string;
    urlEncode(s: string): string;
    urlDecode(s: string): string;
    htmlEntityDecode(s: string): string;
    pad(s: string, len: number, padChar?: string, side?: "left" | "right" |
"both"): string;
    lpad(s: string, len: number, padChar?: string): string;
    rpad(s: string, len: number, padChar?: string): string;
    sprintf(fmt: string, ...args: unknown[]): string;
    printf(fmt: string, ...args: unknown[]): void;
    normalizeNewlines(s: string, style: "lf" | "crlf" | "cr"): string;
  };

  path: {
    dirname(p: string): string;
    basename(p: string, suffix?: string): string;
  };

  hash: {
    md5(data: string): string;
    sha1(data: string): string;
    sha256(data: string): string;
    sha384(data: string): string;
    sha512(data: string): string;
    sha3_256(data: string): string;
    sha3_512(data: string): string;
    blake3(data: string): string;
    crc32(data: string): string;
  };

  compression: {
    algos(): string[];
    compress(
      algo: "gzip" | "deflate" | "zlib" | "bzip2" | "zstd" | "brotli" |
"lz4" | "xz" | "snappy",
      data: string | ArrayBuffer | Uint8Array,
    ): Promise<Uint8Array>;
    decompress(
      algo: "gzip" | "deflate" | "zlib" | "bzip2" | "zstd" | "brotli" |
"lz4" | "xz" | "snappy",
      data: string | ArrayBuffer | Uint8Array,
    ): Promise<Uint8Array>;
  };
```

```
  barcode: {
    formats(): string[];
    decodableFormats(): string[];
    encode(
      format: "qr" | "datamatrix" | "aztec" | "pdf417"
            | "code128" | "code39" | "codabar"
            | "ean13" | "ean8" | "upca",
      data: string,
      opts?: { width?: number; height?: number; quietZone?: boolean |
number },
    ): Promise<Uint8Array>;
    decode(
      data: Uint8Array | ArrayBuffer | string,        // PNG / JPEG / WebP
image bytes
      format?: "qr" | "datamatrix" | "aztec"
             | "code128" | "code39" | "code93" | "codabar"
             | "ean13" | "ean8" | "upca" | "upce" | "itf",
    ): Promise<{ format: string; text: string }>;
  };

  text: {
    detect(data: string | ArrayBuffer | Uint8Array): Promise<{
      charset: string;
      confidence: number;
      language?: string;
      candidates: Array<{ charset: string; confidence: number; language?:
string }>;
    }>;
    decode(data: string | ArrayBuffer | Uint8Array, charset: string):
Promise<string>;
    encode(text: string, charset: string): Promise<Uint8Array>;
  };

  checkdigit: {
    algos(): string[];
    validate(
      algo: "luhn" | "isbn10" | "isbn13" | "ean13" | "ean8" | "upca",
      input: string,
    ): boolean;
    compute(
      algo: "luhn" | "isbn10" | "isbn13" | "ean13" | "ean8" | "upca",
      partial: string,
    ): string;
    inspect(
      algo: "luhn" | "isbn10" | "isbn13" | "ean13" | "ean8" | "upca",
      input: string,
    ): {
      algo: string;
      input: string;
```

```
      valid: boolean;
      given: string;
      computed: string;
    };
  };

  archive: {
    create(
      destPath: string,
      sources: Array<string | { path: string; name?: string }>,
    ): Promise<{ path: string; format: string; entries: string[]; bytes?:
number }>;
    extract(
      archivePath: string,
      destDir: string,
      opts?: { overwrite?: boolean },
    ): Promise<{ path: string; format: string; dest: string; entries:
string[] }>;
  };

  diff: {
    compare(
      a: string | ArrayBuffer | Uint8Array,
      b: string | ArrayBuffer | Uint8Array,
      opts?: { context?: number; fromFile?: string; toFile?: string },
    ): Promise<{
      identical: boolean;
      binary: boolean;
      added: number;
      removed: number;
      diff: string;
      format: "unified";
    }>;
  };

  jq: {
    query(data: unknown, filter: string): Promise<unknown>;
    queryAll(data: unknown, filter: string): Promise<unknown[]>;
  };

  net: {
    tcp(target: string, opts?: { timeout?: number; port?: string }):
Promise<{
      host: string; port: number; ip: string; latencyMs: number;
    }>;
    dns(host: string, opts?: { types?: ("a" | "aaaa" | "mx" | "txt" |
"cname" | "ns")[] }): Promise<{
      a?: string[]; aaaa?: string[];
      mx?: { preference: number; host: string }[];
```

```
      txt?: string[]; cname?: string; ns?: string[];
    }>;
    tls(target: string, opts?: { timeout?: number }): Promise<{
      cn: string; issuer: string;
      notBefore: string; notAfter: string;
      daysRemaining: number;
      dnsNames: string[];
      serialNumber: string;
      fingerprintSha256: string;
    }>;
    ntp(host: string, opts?: { timeout?: number; port?: number }):
Promise<{
      serverTime: string;
      offsetMs: number;
      rttMs: number;
      stratum: number;
      referenceTime: string;
      rootDelayMs: number;
      rootDispersionMs: number;
    }>;
    whois(domain: string, opts?: { timeout?: number }): Promise<{
      raw: string;
      domain?: {
        name: string; punycode?: string;
        whoisServer?: string;
        nameServers?: string[]; status?: string[];
        dnssec?: boolean;
        createdDate?: string; updatedDate?: string; expirationDate?:
string;
      };
      registrar?: { name?: string };
    }>;
    ping(host: string, opts?: {
      count?: number;          // default 4
      timeout?: number;        // ms; default 5000
      mode?: "tcp" | "icmp";   // default "tcp"
      port?: string;           // tcp mode; default "80"
    }): Promise<{
      host: string; ip: string; mode: string;
      sent: number; received: number; lossPercent: number;
      minMs: number; avgMs: number; maxMs: number;
    }>;
    smtp(host: string, opts?: {
      port?: string;           // default "25"
      timeout?: number;        // ms; default 10000
      ehloName?: string;       // default "localhost"
    }): Promise<{
      host: string; port: string; banner: string; ehloDomain: string;
      extensions: string[]; starttls: boolean;
```

```
      authMechanisms: string[]; sizeLimit: number;
    }>;
    wss(url: string, opts?: {
      timeout?: number;        // ms; default 10000
      ping?: boolean;          // measure ping/pong RTT; default true
    }): Promise<{
      url: string; connected: boolean; subprotocol: string;
      status: number; handshakeMs: number; pingMs: number;  // pingMs -1 if
skipped/failed
    }>;
  };

  netstatus: {
    check(host: string, opts?: { port?: string; timeout?: number }):
Promise<{
      host: string; port: string; elapsedMs: number; reachable: boolean;
      dns:  { ok: boolean; ips: string[]; error?: string };
      tcp:  { ok: boolean; latencyMs: number; error?: string };
      tls:  { ok: boolean; daysRemaining: number; error?: string };
      http: { ok: boolean; status: number; error?: string };
    }>;
  };

  redis: {
    open(url: string): Promise<{                 //
redis://[:pass@]host:port/db
      do(command: string, ...args: unknown[]): Promise<unknown>;  //
arbitrary RESP command; null on missing key
      ping(): Promise<string>;
      close(): Promise<void>;
    }>;
  };

  memcached: {
    open(addr: string): Promise<{                // host:port
      get(key: string): Promise<string | null>;  // null on cache miss
      set(key: string, value: string | Uint8Array, expirySeconds?: number):
Promise<void>;
      delete(key: string): Promise<boolean>;      // true if the key
existed
    }>;
  };

  ldap: {
    open(url: string, opts?: { bindDN?: string; password?: string }):
Promise<{
      rootDSE(): Promise<Record<string, unknown>>;            // server
metadata entry
      search(baseDN: string, filter: string, attrs?: string[]):
```

```
Promise<Array<Record<string, unknown>>>;
      close(): Promise<void>;
    }>;
  };

  dict: {
    define(host: string, word: string, opts?: { database?: string; port?:
string; timeout?: number }): Promise<{
      word: string; found: boolean;
      definitions: Array<{ db: string; dbName: string; text: string }>;
    }>;
    match(host: string, word: string, opts?: { database?: string;
strategy?: string; port?: string; timeout?: number }): Promise<{
      word: string; matches: Array<{ db: string; word: string }>;
    }>;
  };

  ai: {
    providers(): string[];                       // detected CLIs on PATH,
preference order
    send(opts: {
      prompt: string;
      provider?: string;                          // default: first
detected
      system?: string;
      context?: string;
      timeout?: number;                           // ms; default 120000
    }): Promise<{ provider: string; output: string; exitCode: number }>;
  };

  browser: {
    open(): Promise<{                            // stateful HTTP session
      setUserAgent(ua: string): void;
      setHeader(name: string, value: string): void;
      get(url: string): Promise<{ status: number; ok: boolean; headers:
Record<string, string>; body: string; url: string }>;
      post(url: string, body?: string): Promise<{ status: number; ok:
boolean; headers: Record<string, string>; body: string; url: string }>;
      cookies(url: string): Promise<Array<{ name: string; value: string
}>>;
    }>;
  };

  exec: {
    shell(
      cmd: string | string[],
      opts?: {
        cwd?: string;
        env?: Record<string, string>;
```

```
        timeout?: number; // ms; default 30000
        stdin?: string;
      },
    ): Promise<{
      stdout: string;
      stderr: string;
      exitCode: number;
      success: boolean;
      durationMs: number;
    }>;
    http(
      method: string,
      url: string,
      opts?: {
        headers?: Record<string, string>;
        body?: string;
        timeout?: number;       // ms; default 30000
        follow?: boolean;       // -L
        insecure?: boolean;     // -k
        backend?: "auto" | "recon" | "curl"; // default "auto"
      },
    ): Promise<{
      status: number;
      headers: Record<string, string>;
      body: string;
      durationMs: number;
      backend: "recon" | "curl";
    }>;
  };

  git: {
    branch(opts?: { cwd?: string }): Promise<{
      current: string;           // branch name; empty on detached HEAD
      detached: boolean;
      all: string[];             // local branches
    }>;
    isClean(opts?: { cwd?: string }): Promise<boolean>;
    revParse(rev: string, opts?: { cwd?: string }): Promise<string>;
    status(opts?: { cwd?: string }): Promise<Array<{
      path: string;
      indexStatus: string;       // single-char porcelain code
      workingStatus: string;
    }>>;
    add(paths: string | string[], opts?: { cwd?: string }): Promise<{
paths: string[] }>;
    commit(message: string, opts?: { cwd?: string; allowEmpty?: boolean }):
Promise<{ sha: string }>;
    log(opts?: { cwd?: string; limit?: number; revRange?: string }):
Promise<Array<{
```

```
      sha: string;
      shortSha: string;
      author: string;
      email: string;
      timestamp: number;         // unix seconds
      subject: string;
    }>>;
    diffStat(opts?: { cwd?: string; revRange?: string }): Promise<{
      files: number;
      insertions: number;
      deletions: number;
    }>;
    runText(args: string[], opts?: { cwd?: string }): Promise<{
      stdout: string;
      stderr: string;
      exitCode: number;          // surfaced as data — does NOT throw
    }>;
  };

  preg: {
    match(pattern: string, subject: string): {
      match: string;
      groups: string[];     // empty string for an optional group that
didn't match
      index: number;
    } | null;
    matchAll(pattern: string, subject: string): Array<{
      match: string;
      groups: string[];
      index: number;
    }>;
    replace(pattern: string, replacement: string, subject: string): string;
  };

  preg2: {                                      // PCRE engine
(lookaround, backrefs)
    match(pattern: string, subject: string): {
      match: string;
      groups: string[];
      index: number;
    } | null;
    matchAll(pattern: string, subject: string): Array<{
      match: string;
      groups: string[];
      index: number;
    }>;
    replace(pattern: string, replacement: string, subject: string): string;
  };
```

```
  jwt: {
    sign(
      claims: Record<string, unknown>,
      secret: string,                            // raw bytes for HS*; PEM-
encoded private key for RS*/PS*/ES*/EdDSA
      opts?: {
        algorithm?:
          | "HS256" | "HS384" | "HS512"          // HMAC
          | "RS256" | "RS384" | "RS512"          // RSA-PKCS1v1.5
          | "PS256" | "PS384" | "PS512"          // RSA-PSS
          | "ES256" | "ES384" | "ES512"          // ECDSA
          | "EdDSA";                             // Ed25519
      },                                         // default "HS256"
    ): string;
    view(token: string): {
      header: Record<string, unknown>;
      payload: Record<string, unknown>;
      signature: string;                         // raw base64url, no
padding
    };
    validate(
      token: string,
      secret: string,                            // raw bytes for HS*; PEM-
encoded public key (or certificate) for asymmetric
      opts?: {
        algorithm?:
          | "HS256" | "HS384" | "HS512"
          | "RS256" | "RS384" | "RS512"
          | "PS256" | "PS384" | "PS512"
          | "ES256" | "ES384" | "ES512"
          | "EdDSA";
        audience?: string;
        issuer?: string;
      },
    ): { valid: true; claims: Record<string, unknown> }
     | { valid: false; reason: string };
  };

  encrypt: {
    keygen(): { publicKey: string; privateKey: string };
// age X25519
    keygenPgp(opts?: { name?: string; email?: string }):
// PGP keypair (armored blocks)
      { publicKey: string; privateKey: string };
    encrypt(
      data: string | Uint8Array | ArrayBuffer,
      recipients: string | string[],          // age1... public keys
      opts?: { armored?: boolean },           // default false → binary age
format
```

```
    ): Uint8Array;
    decrypt(
      ciphertext: string | Uint8Array | ArrayBuffer,
      identities: string | string[],          // AGE-SECRET-KEY-1...
private keys
    ): Uint8Array;                             // auto-detects binary vs
armored
    rekey(
      ciphertext: string | Uint8Array | ArrayBuffer,
      oldIdentities: string | string[],
      newRecipients: string | string[],
      opts?: { armored?: boolean },           // default: preserve input
format
    ): Uint8Array;
    detectBackend(input: string):
      | { backend: "age";     kind: "public" | "private" }
      | { backend: "pgp";     kind: "public" | "private" }
      | { backend: "unknown" };
  };

  sqlite: {
    open(path: string): Promise<{               // ":memory:" or a
filesystem path
      exec(sql: string, ...params: unknown[]): Promise<{
        rowsAffected: number;
        lastInsertId: number;
      }>;
      query(sql: string, ...params: unknown[]):
Promise<Array<Record<string, unknown>>>;
      queryValue(sql: string, ...params: unknown[]): Promise<unknown>;  //
first column of first row, or null
      begin(): Promise<{                        // transaction handle
        exec(sql: string, ...params: unknown[]): Promise<{ rowsAffected:
number; lastInsertId: number }>;
        query(sql: string, ...params: unknown[]):
Promise<Array<Record<string, unknown>>>;
        queryValue(sql: string, ...params: unknown[]): Promise<unknown>;
        commit(): Promise<void>;
        rollback(): Promise<void>;
      }>;
      prepare(sql: string): Promise<{           // compiled-once statement
handle
        exec(...params: unknown[]): Promise<{ rowsAffected: number;
lastInsertId: number }>;
        query(...params: unknown[]): Promise<Array<Record<string,
unknown>>>;
        queryValue(...params: unknown[]): Promise<unknown>;
        close(): Promise<void>;
      }>;
```

```
      close(): Promise<void>;
    }>;
  };

  gh: {
    authStatus(): Promise<{
      authenticated: boolean;
      user: string;              // login or empty
      raw: string;               // login on success, stderr / probe note
on failure
    }>;
    prList(opts?: {
      cwd?: string;
      state?: "open" | "closed" | "merged" | "all";
      limit?: number;            // default 30
      author?: string;
    }): Promise<Array<{
      number: number;
      title: string;
      state: string;
      author: string;            // login (flattened from gh's wrapper)
      headRefName: string;
      baseRefName: string;
      url: string;
      createdAt: string;
      updatedAt: string;
    }>>;
    repoView(
      repo?: string,             // "owner/name"; omit to use cwd's repo
      opts?: { cwd?: string },
    ): Promise<{
      name: string;
      owner: string;             // login (flattened)
      description: string;
      url: string;
      defaultBranch: string;     // flattened from defaultBranchRef.name
      visibility: string;
    }>;
  };

  email: {
    spf(domain: string): Promise<
      | { present: false }
      | {
          present: true;
          record: string;
          mechanisms: string[];
          allPolicy: "pass" | "fail" | "softfail" | "neutral" | "";
        }
```

```
  >;
  dmarc(domain: string): Promise<
    | { present: false }
    | {
        present: true;
        record: string;
        tags: Record<string, string>;
        policy?: string;
        subdomain?: string;
        percent?: string;
        rua?: string;
        ruf?: string;
      }
  >;
  mtaSts(domain: string): Promise<
    | { present: false }
    | {
        present: true;
        record: string;
        txt: { v: string; id: string };
        policy?: {
          version?: string;
          mode?: "enforce" | "testing" | "none" | string;
          mx?: string[];
          maxAge?: number | string;
        };
        policyError?: string;
      }
  >;
  tlsRpt(domain: string): Promise<
    | { present: false }
    | {
        present: true;
        record: string;
        tags: Record<string, string>;
        rua?: string;
      }
  >;
  bimi(domain: string, opts?: { selector?: string }): Promise<
    | { present: false; selector: string }
    | {
        present: true;
        selector: string;
        record: string;
        tags: Record<string, string>;
        l?: string;
        a?: string;
      }
  >;
```

```
    all(domain: string): Promise<{
      domain: string;
      spf:    unknown;
      dmarc:  unknown;
      mtaSts: unknown;
      tlsRpt: unknown;
      bimi:   unknown;
    }>;
  };
};
```

Examples:

```
api.runtime.log("hello", 1 + 2);
api.runtime.assert.equal(api.runtime.time.nowMs() > 0, true);

const r = await api.net.http.get("https://example.com");
api.runtime.assert.ok(r.status === 200, `expected 200, got ${r.status}`);

await api.runtime.time.sleep(50);
const home = api.runtime.env.get("HOME") ?? "(none)";

api.runtime.log(api.crypto.hash.sha256("abc"));
// → ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
```

`api.net.http.request(method, url, opts?)` is the full-featured client beyond `get` / `post`: per-call `headers`, `body`, `timeout` (default 30s), `retry` (re-attempts on transport errors and 5xx — never 4xx, which are deterministic; linear backoff capped at 1s), `follow` (redirect control; default true), and basic-auth `username` / `password`. The result is `{ status, ok, headers, body, url }` — `ok` is `status` in [200,400), `headers` is a lower-cased name→value map, `url` is the final URL after redirects. HTTP 4xx / 5xx don't throw (surfaced via `status` / `ok`); transport errors and context deadline do.

HTTP bindings use `net/http` with a 5-second default per-request timeout and surface real `Promise<…>` values through the event loop. They are *not* mockable from JS — they go to the real network.

Compression bindings (`api.format.compression.*`) cover nine pure-Go algorithms behind a uniform interface. Inputs are either strings (interpreted as UTF-8 byte sequences) or any `ArrayBuffer` / `Uint8Array`; outputs are `Uint8Array` — goja's representation of a Go `[]byte` return is a `Uint8Array`, so scripts can read `.length` and iterate directly without an `ArrayBuffer` view. Every algorithm round-trips:

Algorithm	Library
`gzip` / `deflate` / `zlib`	stdlib `compress/*`

Algorithm	Library
`bzip2`	stdlib `compress/bzip2` for read; `github.com/dsnet/compress/bzip2` for write
`zstd`	`github.com/klauspost/compress/zstd`
`brotli`	`github.com/andybalholm/brotli`
`lz4`	`github.com/pierrec/lz4/v4`
`xz`	`github.com/ulikunitz/xz`
`snappy`	`github.com/golang/snappy`

`api.format.compression.algos()` returns the supported names so scripts can iterate without hard-coding the list. Unknown algorithm names throw a clear error rather than silently returning empty bytes.

Barcode encoders (`api.format.barcode.*`) cover ten symbologies through one `encode(format, data, opts?)` entry point, all backed by the pure-Go `github.com/boombuler/barcode` toolkit. Output is a PNG payload as `Uint8Array` ; size defaults to `256x256` for 2D codes (QR / DataMatrix / Aztec) and `400x120` for linear ones, overridable via `opts.width` / `opts.height` . `opts.quietZone` (`true` for a default margin, or a pixel count) pads the rendered bars with a white border — required for EAN / UPC to decode (see the decode quirks below).

- 2D: `qr` (medium error correction, auto mode), `datamatrix` , `aztec` (33% ECC, auto layers), `pdf417` (security level 5).
- Linear: `code128` , `code39` (Mod-43 checksum, ASCII-only), `codabar` .
- Retail: `ean13` , `ean8` , `upca` — content must be numeric and the right length; `boombuler/ean.Encode` dispatches by length so all three variants share the same encoder.

`api.format.barcode.formats()` returns the supported encode names; `api.format.barcode.decodableFormats()` returns the (slightly different) decode set.

Decoding (`api.format.barcode.decode(data, format?)`) accepts PNG / JPEG / WebP image bytes and reaches into `github.com/makiuchi-d/gozxing` . Returns `{ format, text }` — `format` is the snake-case label the encoder accepts, so round-trips line up. WebP support comes via a blank-import of `golang.org/x/image/webp` that registers the decoder with stdlib's `image.Decode` .

- **With a `format` hint**: only that reader runs. Fast, and a mismatch surfaces a clear error rather than a silent miss.
- **Without a hint**: every reader in a priority list runs (2D formats first; the more-constrained 1D readers — UPC/EAN — before the permissive ones — Code 128 / Codabar). First success wins; if no reader matches, the call throws with "no barcode recognised".

Decode quirks worth knowing — these are properties of the underlying libraries, not sercon bugs:

- **`pdf417` isn't decodable.** gozxing v0.1.1 has no PDF417 reader; pass it as a hint and the call throws "unsupported format". The encoder still emits PDF417 just fine.
- **EAN / UPC need a quiet zone.** Per ISO/IEC 15420 + 15418, retail symbologies require white margin on both sides of the bars. boombuler's encoder emits them edge-to-edge, so a bare `barcode.encode("ean13", x) → barcode.decode(...)` round-trip fails on the decode step. Pass **`opts.quietZone`** to `encode` to fix it: `true` adds a default margin (10% of the width, floored at 10px), or pass a pixel count for an explicit margin. Real-world EAN scanners need that margin too, so it's good practice for any EAN/UPC you'll actually print.
- **Code 39 decoded text ends with the Mod-43 checksum char.** boombuler's encoder appends it; gozxing returns it.
- **Codabar wraps payload in start/stop chars** like `A...A` on encode; gozxing strips them on decode.

Charset bindings (`api.text.*`) cover detection plus byte-side round-tripping:

- **`detect(data)`** — runs [`saintfish/chardet`] over the input and returns the top guess (`charset`, `confidence` on a 0–100 scale, optional `language` hint) plus the full candidate list. Input is bytes (string, `ArrayBuffer`, or `Uint8Array`).
- **`encode(text, charset)`** — converts a UTF-8 string to bytes in the target encoding. Characters with no representation in the target encoding cause the encoder to error out rather than silently lose them; pre-process the input yourself if you want lossy behaviour.
- **`decode(data, charset)`** — the inverse: bytes-in-charset → UTF-8 string.

Charset names follow the WHATWG Encoding Living Standard aliases that `golang.org/x/text/encoding/htmlindex.Get` accepts — `UTF-8`, `ISO-8859-1`, `Windows-1252`, `Shift_JIS`, `GBK`, `GB18030`, `Big5`, `EUC-JP`, `EUC-KR`, etc., plus all documented aliases. Unknown names throw a clear error.

Check-digit bindings (`api.format.checkdigit.*`) verify and compute the trailing check digit of common numeric codes. All three members are synchronous — the algorithms are small bits of modular arithmetic, no I/O or library dependency. Supported algorithms (returned by `algos()`):

Algorithm	Input length	Notes
`luhn`	any ≥ 2	Credit cards, IMEI, social-security-style IDs. Mod-10 with right-to-left doubling.
`isbn10`	10	Last position may be `X` (= 10). Mod-11 with weighted sum.
`isbn13`	13	Alias of `ean13` — same algorithm; the 978 / 979 prefix is the only thing that distinguishes ISBN-13 syntactically.

Algorithm	Input length	Notes
`ean13`	13	Mod-10, weights `1, 3, 1, 3, …` from position 0.
`ean8`	8	Mod-10, weights `3, 1, 3, 1, …`.
`upca`	12	Mod-10, weights `3, 1, 3, 1, …`.

`validate(algo, input)` returns a plain boolean — non-digit characters, wrong length, and unknown algorithm names all surface as `false` so scripts can run quick presence checks without try/catch. `compute(algo, partial)` takes the input *without* its check digit and returns just that digit (or `"X"` for ISBN-10 position 10). `inspect(algo, input)` returns the union of both views — useful when you want to display the diagnosis to a human.

Archive bindings (`api.fs.archive.*`) handle the three formats stdlib provides without external deps: `.zip`, `.tar`, and `.tar.gz` (also `.tgz`). Format is inferred from the destination's extension on `create` and from the source path on `extract`.

- **`create(destPath, sources)`** — sources is an array of either bare paths (the basename is used as the in-archive name) or `{ path, name }` objects (override the in-archive name). Directory sources are recursed; the directory's basename becomes the archive subdir and the rest of the tree is added relative to it. Returns the count + list of entries and the output file size.
- **`extract(archivePath, destDir, opts?)`** — unpacks into destDir. `opts.overwrite` (default `false`) controls whether existing files at the destination are clobbered; on `false` a collision errors out (`os.ErrExist`-shaped). Every entry name is run through a zip-slip / tar-slip guard that rejects absolute paths, `..` segments, and anything that would resolve outside destDir.

Diff bindings (`api.text.diff.compare(a, b, opts?)`) produce a unified diff between two text inputs via `github.com/pmezard/go-difflib`. Inputs are strings (UTF-8) or any byte sequence (`ArrayBuffer` / `Uint8Array`). The result reports:

- `identical` — byte-equal inputs short-circuit; `diff` is empty.
- `binary` — either input has a NUL byte in its first 8 KB. A unified diff of binary content isn't useful so `diff` is empty.
- `added` / `removed` — body-only `+` / `-` line counts (file headers are excluded so the numbers match `git diff --shortstat`).
- `diff` — the unified diff text, or empty when `identical` or `binary` is `true`.
- `format` — always `"unified"` today; reserved for future alternatives (`"side-by-side"`, `"summary"`, …).

`opts.context` (default 3) is the context-line count; `opts.fromFile` / `opts.toFile` (default `"a"` / `"b"`) set the labels in the diff headers.

JSON-query bindings (`api.text.jq.*`) run jq filters over JS data via `github.com/itchyny/gojq`, a pure-Go re-implementation. The filter syntax is the same as command-line jq.

- **`query(data, filter)`** — runs the filter and returns the first emitted value (or `null` when the filter emits nothing).
- **`queryAll(data, filter)`** — drains the result iterator and returns every emitted value as an array.

Data passes through goja's `Export` as `map[string]any` / `[]any` trees, which gojq's runtime accepts directly. There's a small normalisation pass on the way in that converts all sized integer types (`int64`, `int32`, etc.) to plain `int` and `float32` to `float64` — gojq's arithmetic dispatch only knows the two canonical numeric kinds, and goja exports every JS-side integer as `int64`. Without normalisation a query as simple as `.users[].age` would panic in gojq.

Parse errors and runtime errors (type mismatches, division by zero, …) both surface as JS exceptions. Use jq's optional access operator `?` (e.g. `.does.not.exist?`) to suppress missing-path errors and get `null` back instead.

Aggregate connectivity (`api.net.netstatus.check(host, opts?)`) is an orchestration layer over the lower-level probes — it runs DNS, TCP, TLS, and HTTP against one host **concurrently** (fan-out via a WaitGroup) and folds them into a single status object. Each sub-probe carries its own `ok` flag and, on failure, an `error` string — individual failures are data, not a thrown error, so the result is always a complete snapshot. `reachable` is the AND of `dns.ok` and `tcp.ok` (name resolves + connection opens); TLS / HTTP are reported but don't gate it (a plain-HTTP host or one with an expired cert is still "reachable"). Only a missing host argument throws. No new library — it composes `net` / `crypto/tls` / `net/http` directly.

Redis (`api.db.redis.open(url)`) is a stateful-handle binding over `redis/go-redis/v9` (the official client). `open` parses a `redis://[:password@]host:port/db` URL and PINGs to surface a bad address up front, then resolves to `{ do, ping, close }`. `do(command, ...args)` runs an arbitrary RESP command — `do("SET", "k", "v")`, `do("LRANGE", "l", "0", "-1")` — so the binding stays small instead of mirroring hundreds of methods. Replies map the obvious way (bulk strings → string, integers → number, arrays → array); a nil reply (missing key) becomes JS `null` rather than throwing, so `await r.do("GET", "absent")` is `null`. Redis-level errors (WRONGTYPE, unknown command) throw. Offline-testable via `alicebob/miniredis`.

Memcached (`api.db.memcached.open(addr)`) is a stateful-handle binding over `bradfitz/gomemcache` (the de facto standard client). `addr` is `host:port`; gomemcache pools connections lazily so there's no PING-on-open (the first op surfaces a bad address) and no close. Resolves to `{ get, set, delete }`: `get` returns the stored string or `null` on a cache miss; `set(key, value, expirySeconds?)` stores bytes (0 / omitted = never expire); `delete` returns `true` if the key existed, `false` on a miss. Server errors throw.

LDAP (`api.db.ldap.open(url)`) is a stateful-handle binding over `go-ldap/v3`. Dials `ldap://host:port` (or `ldaps://`), does an anonymous bind by default (or a simple bind with `opts.bindDN` / `opts.password`), and resolves to `{ rootDSE, search, close }`. `rootDSE()` reads the server's anonymous metadata entry (naming contexts, supported controls, vendor); `search(baseDN, filter, attrs?)` runs a subtree search and returns `{ dn, <attr>: [values] }` per entry (attributes stay arrays since LDAP is multi-valued). A read / inspection surface — no modify / add / delete.

DICT (`api.db.dict.*`) is an RFC 2229 dictionary-server client. No maintained pure-Go DICT library exists, so the protocol is hand-rolled over `net/textproto` (a simple line-based status-code protocol, much like SMTP). `define(host, word, opts?)` returns `{ word, found, definitions: [{ db, dbName, text }] }`; a word with no entry resolves `found: false` (not an error). `match(host, word, opts?)` returns words matching under `opts.strategy` (default `prefix`). Both are one-shot (connect → query → QUIT); `opts.database` (default `*` = all), `opts.port` (default 2628).

AI agents (`api.tools.ai.*`) shell out to a coding-assistant CLI. `providers()` lists which of `claude` / `codex` / `copilot` / `gemini` are on PATH (preference order); `send(opts)` runs a one-shot prompt through one of them — `opts.provider` picks explicitly, else the first detected. `opts.system` / `opts.context` are prepended to the prompt (the portable common denominator across CLIs with different system-prompt flags). The argv builder is a pure function (`buildAIArgv`) so the provider invocations are unit-testable without the CLIs installed. Returns `{ provider, output, exitCode }`; a non-zero exit is surfaced as data (the model CLI may exit non-zero with a useful message), while a missing provider and context deadline throw. This is the options-object shape rather than a builder chain — the idiomatic JS equivalent, and it sidesteps threading a mutable builder handle through goja. **Library:** `os/exec` (stdlib).

Browser sessions (`api.net.browser.open()`) give a stateful HTTP client — an automatic cookie jar (`net/http/cookiejar` with the `golang.org/x/net/publicsuffix` list, so cookies scope correctly across subdomains) plus default headers replayed on every request. A second stateful-handle binding in the same shape as `api.db.sqlite`: `open()` resolves to `{ setUserAgent, setHeader, get, post, cookies }`. A login POST followed by a GET replays the session cookie without the script touching it; `cookies(url)` inspects the jar (handy for asserting a login set what you expect). `get` / `post` return the same `{ status, ok, headers, body, url }` shape as `api.net.http.request`. No explicit close — the session is GC'd with the handle.

Subprocess bindings (`api.tools.exec.*`) wrap Go's `os/exec`. `shell` is the only entry today; `http` (recon-with-curl-fallback) and `git` / `gh` wrappers ride on top of it in later 0.4.x cuts.

- **`shell(cmd, opts?)`** — runs a subprocess and waits for it to exit. `cmd` is either a **string** (passed verbatim to `/bin/sh -c` on Unix or `cmd /C` on Windows so pipes, redirects, and globs work as typed) or a **`string[]`** (treated as argv, no shell — use this when arguments could be re-interpreted by the shell). Returns `{ stdout, stderr, exitCode, success, durationMs }`.

  Options: `cwd` sets the working directory; `env` is merged on top of the parent process environment (not a replacement); `timeout` is in milliseconds and defaults to **30 000 ms**;

    `stdin` is piped to the child's standard input.

    Process-start failures (host binary not on PATH, permission denied) throw. Context deadline (`timeout`) and engine cancellation throw too — the subprocess never got to choose its own exit code, so it isn't reasonable to surface a fake one. Non-zero exits do **not** throw: `success: false` + the real `exitCode` is what callers want for the usual "ran the linter, expected to be told it failed" flow.

- **`http(method, url, opts?)`** — curl-compatible HTTP client routed through `recon` (preferred) with `curl` as a fallback. Returns `{ status, headers, body, durationMs, backend }`. The choice between the two binaries is normally invisible to scripts — both implement the same curl-style flag set we use here (`-X`, `-H`, `-D`, `--data-binary`, `-L`, `-k`, `-s`). Pick a specific backend with `opts.backend: "recon" | "curl"` when you need to.

    Implementation notes: response headers are dumped to a temp file via `-D <path>` and parsed back, rather than relying on `-i`'s body-stream format (recon's `-i` is verbose-debug style with `< ` prefixes, incompatible with curl's wire-format `-i`). Request bodies are materialised to a temp file and passed via `--data-binary @<path>` so CR / LF are preserved regardless of backend. Header names in the returned map are lower-cased. On redirect chains (`opts.follow: true`), the *last* response wins — the intermediate 3xx blocks are skipped.

    HTTP 4xx / 5xx do **not** throw — they're a normal HTTP outcome and callers branch on `status`. Process-start failures, transport errors (DNS, connection refused, TLS handshake), and context deadline / cancellation throw. The error message includes the backend's stderr when present.

Git bindings (`api.tools.git.*`) shell out to the host `git` binary; no pure-Go alternative (`go-git` is heavier and would drift from the user's installed git). Every binding accepts an `opts.cwd` so a single engine can work across multiple checkouts.

- **`branch(opts?)`** — Reports the current branch (empty string when HEAD is detached) plus every local branch from `for-each-ref refs/heads`. `detached` is true exactly when `symbolic-ref --short HEAD` exited non-zero, which is git's own signal for the state.
- **`isClean(opts?)`** — Convenience boolean over `git status --porcelain`. True when the porcelain output is empty.
- **`revParse(rev, opts?)`** — Returns the 40-char SHA for `rev`. Invalid refs throw with git's stderr message included.
- **`status(opts?)`** — Parses `--porcelain` v1 output into `{ path, indexStatus, workingStatus }`. The two single-char status codes are the raw porcelain codes (`M`, `A`, `D`, `R`, `?`, ...) so scripts can dispatch on them without re-parsing strings.
- **`add(paths, opts?)`** — Stages one path (string) or a list (string[]). `--` is inserted between `add` and the paths so values that look like flags work too. Returns the resolved paths array.

- **`commit(message, opts?)`** — Creates a new commit; the post-commit HEAD SHA is returned. `allowEmpty:true` toggles `--allow-empty`. An empty message throws before spawning.
- **`log(opts?)`** — Returns `limit` (default 50) most-recent commits in `revRange` (default `HEAD`). Format string `%H%x09%h%x09%an%x09%ae%x09%at%x09%s` tab-separates SHA, short SHA, author name, email, unix timestamp, and subject. Newlines and tabs in subjects are already collapsed by git so the parse stays one-line-per-commit.
- **`diffStat(opts?)`** — Aggregates `git diff --shortstat` into `{ files, insertions, deletions }`. Default range is `HEAD~1..HEAD`. A pure-add or pure-delete diff returns zero for the missing side instead of throwing.
- **`runText(args, opts?)`** — Escape hatch. Surfaces `{ stdout, stderr, exitCode }` for any `git <args>` invocation. Non-zero exits do **not** throw — that's the whole point of having a generic wrapper; callers branch on `exitCode`. Spawn failures and context cancellation still throw.

GitHub-CLI bindings (`api.tools.gh.*`) wrap the `gh` binary. They respect whatever authentication state `gh auth` is already in; we don't try to swap accounts or manage tokens. Every call uses `gh --json` so the result is structured rather than parsed out of human-readable text.

- **`authStatus()`** — Probe whether gh is installed *and* authenticated. Under the hood this runs `gh api user --jq .login` (machine-friendly: just the login on success, a clear error otherwise) rather than `gh auth status` (multi-line human report). Missing-gh and unauthenticated-session both resolve with `{ authenticated: false, …}` rather than throwing — the whole point of a status probe is that the script branches on it. Only context cancellation throws.
- **`prList(opts?)`** — Lists pull requests on the repo identified by `opts.cwd` (or the engine's working directory). Defaults: open state, limit 30. `gh`'s `author: { login, … }` wrapper is flattened to just the login string so scripts can compare directly.
- **`repoView(repo?, opts?)`** — Returns metadata about a repo. With no argument it asks `gh` about the cwd's repo (works from inside a checkout); pass `"owner/name"` to look up any repo `gh` has access to. Two convenience flattenings are applied: `owner` is a login string (not `{login, …}`), and `defaultBranch` is the branch name (not `defaultBranchRef.name`). Empty repos resolve with `defaultBranch: ""` rather than `undefined`.

Regex bindings (`api.text.preg.*`) accept PHP-style `/pattern/flags` delimited input but run on Go's stdlib `regexp` (RE2). The "preg" naming is a deliberate homage; the semantics are RE2's. That means **no backreferences inside patterns**, **no lookahead / lookbehind**, and **no possessive quantifiers** — scripts that need those should reach for goja's native `RegExp` (ECMAScript-flavoured) or wait for a dedicated PCRE-compatible binding (`regexp2` is on the backlog).

Supported flags: `i` (case-insensitive), `m` (multiline — `^` and `$` match line boundaries), `s` (dotall — `.` matches newline). The PHP flags `u` (Unicode), `U` (default-ungreedy), and `x` (extended whitespace-ignoring) all surface a clear error rather than silently dropping; unknown flags do too. RE2 is UTF-8 by default, so `u` is unnecessary in any case — the error message says so.

- **`match(pattern, subject)`** — Returns `{ match, groups, index }` for the first hit, or `null` when nothing matches. `groups` is every numbered submatch after group 0, in order; an optional group that didn't match surfaces as an empty string (not `undefined`) so the result type stays stable.
- **`matchAll(pattern, subject)`** — Same shape, drained: returns an array of match objects, one per hit.
- **`replace(pattern, replacement, subject)`** — Substitutes via Go's `$1` / `${1}` backreference syntax. PHP's `\1` form is **not** translated — that would conflict with `\\` escapes that are already legitimate in the replacement. Use `$1` / `${1}` as in Go.

PCRE regex bindings (`api.text.preg2.*`) are the heavier-engine sibling of `api.text.preg`. They run on `github.com/dlclark/regexp2`, a port of .NET's regex engine, which **does** support lookahead, lookbehind, backreferences inside patterns, and possessive quantifiers — everything RE2 (and therefore `api.text.preg`) can't do. The API is identical (`match` / `matchAll` / `replace`, the same `/pattern/flags` delimited syntax, the same `{ match, groups, index }` shape) so switching engines is a one-word change. The flag set adds `x` (ignore-pattern-whitespace), which RE2 couldn't honour; `u` / `U` still error (regexp2 is Unicode-aware by default and has no global-ungreedy switch). `replace` uses .NET / regexp2 `$1` / `${1}` substitution syntax.

The trade-off is the reason `api.text.preg` exists at all: regexp2 **backtracks**, so it has no linear-time guarantee. A pathological pattern against adversarial input can blow up exponentially. Use `api.text.preg` (RE2) by default; reach for `api.text.preg2` only when you need a PCRE feature, and keep a timeout around untrusted input.

JWT bindings (`api.crypto.jwt.*`) wrap `golang-jwt/jwt/v5`. The full RFC 7518 algorithm matrix is supported: HMAC (`HS256` / `HS384` / `HS512`), RSA-PKCS1v1.5 (`RS256` / `RS384` / `RS512`), RSA-PSS (`PS256` / `PS384` / `PS512`), ECDSA (`ES256` / `ES384` / `ES512`), and Ed25519 (`EdDSA`). Unsupported algorithm names — including the special `"none"` value, garbage like `"HS999"`, and protocol names that aren't JWT signing algos — throw at `sign` / `validate` time with a named-algorithm error rather than silently falling through.

Key shape. The `secret` parameter takes three forms, picked by shape: HMAC algorithms use the byte string directly; asymmetric algorithms expect a PEM-encoded key (private for `sign`, public — or a certificate the public key can be pulled from — for `validate`); and **any algorithm** accepts a JWK (JSON Web Key) object — a JSON string carrying a `"kty"` member. JWK is detected by the leading `{` + `"kty"` and parsed via `lestrrat-go/jwx/v2/jwk`; the `kty` (`oct` → HMAC bytes, `RSA`, `EC`, `OKP`) determines the key type, so a JWK works with whatever matching `opts.algorithm` you pass. Useful when keys come from a JWKS endpoint or a config file in JWK form. PEM detection is the literal `-----BEGIN` prefix. Both directions of the PEM/bytes cross-check are enforced (JWK input bypasses them — the `kty` is authoritative):

- **PEM secret + HMAC algorithm** throws with a named-algorithm hint ("looks like a PEM-encoded key — set opts.algorithm to RS256 / ES256 / EdDSA / etc."). Without this guard, a script that forgets to set `opts.algorithm` after switching to a PEM key would silently sign an HS256 token using the PEM bytes themselves — that's exactly the algorithm-confusion attack pattern in reverse.
- **Plain bytes + asymmetric algorithm** throws with the inverse hint ("needs a PEM-encoded private/public key but secret is plain bytes"). Without this guard the user would get a cryptic "x509: malformed certificate" deep inside jwt-go.

The cross-check fires at the binding boundary before jwt-go is called, so it's a thrown error rather than a `valid: false` resolution — these are structural input problems, not validation failures.

- **`sign(claims, secret, opts?)`** — Produces a compact-serialised signed JWT. `claims` is passed straight through to jwt-go's `MapClaims`, which recognises the RFC 7519 reserved claims (`exp`, `nbf`, `iat`, `aud`, `iss`, `sub`, `jti`) and lets you layer arbitrary application claims alongside them. Missing reserved claims aren't synthesised — scripts that want `iat` set should compute it explicitly (e.g. `Math.floor(api.runtime.time.nowMs() / 1000)`).

- **`view(token)`** — Decodes the header + payload **without verifying the signature**. Useful for debugging auth flows or surfacing `aud` / `iss` to the user before deciding whether to trust the token. `signature` comes back as the raw base64url-encoded segment (no padding); decode it yourself if you need the bytes. Malformed input — wrong segment count, invalid base64, invalid JSON — throws.

- **`validate(token, secret, opts?)`** — Verifies the signature using `secret` and the algorithm declared in the token's header. Standard-claim validation (`exp`, `nbf`, `iat`) is delegated to jwt-go. `opts.audience` / `opts.issuer` are cross-checked against `aud` / `iss` when set. The contract is **resolve, don't throw, on validation failure**: `{ valid: false, reason: "…" }` for bad signature, expired, audience mismatch, issuer mismatch, *and* algorithm mismatch when `opts.algorithm` is set. Only structural input errors (not a JWT at all, empty secret, empty token, PEM/algorithm cross-check failure) throw — pattern-matching on `valid: false` shouldn't accidentally accept a garbage string.

  Setting `opts.algorithm` is the **algorithm-confusion guard**: an attacker who has the public key bytes can forge an HS256 token by using those bytes as the HMAC secret (the classic JWT exploit). When `opts.algorithm` is set, the parser's `WithValidMethods` whitelist refuses any token whose header declares a different algorithm. Always set `opts.algorithm` when validating asymmetric tokens; the binding doesn't infer it from the key shape because that would make the wrong default silent.

Encryption (`api.crypto.encrypt.*`) supports **two backends** behind one API: age ([`filippo.io/age`](filippo.io/age), the default) and PGP ([`ProtonMail/go-crypto/openpgp`](ProtonMail/go-crypto/openpgp)). `encrypt` / `decrypt` auto-dispatch on the key/ciphertext format — the same classifier `detectBackend` exposes — so scripts use one API regardless of backend. The age round-trip core (keygen / encrypt / decrypt) landed in v0.5.5; armoured output (`opts.armored`) in v0.5.6; `rekey` in v0.5.7; `detectBackend` in v0.5.8; the PGP backend in v0.5.16.

age identities use the X25519 flavour: `age1...` recipients (safe to share) and `AGE-SECRET-KEY-1...` identities (kept secret). PGP keys are ASCII-armored blocks (`-----BEGIN PGP PUBLIC/PRIVATE KEY BLOCK-----`), generated by `keygenPgp`.

- **`keygenPgp(opts?)`** — Generate a PGP keypair (RSA 2048). `opts.name` / `opts.email` populate the primary user ID; both optional. Returns armored `{ publicKey, privateKey }` blocks. When `encrypt` sees a PGP public-key block as a recipient (or `decrypt` sees a PGP private-key identity / a `-----BEGIN PGP MESSAGE-----` ciphertext), it routes through openpgp instead of age. PGP output is always armored; `opts.armored` is ignored on that path. age and PGP recipient sets can't be mixed in one call — the formats are incompatible.

All three members are synchronous: encryption is pure CPU work with a small API surface, matching the call shape of the other crypto bindings (`api.crypto.jwt`, `api.crypto.hash`).

- **`keygen()`** — Generate a fresh X25519 identity. Returns `{ publicKey, privateKey }` as the bech32 strings age writes to disk. Two consecutive calls produce different keys; entropy comes from `crypto/rand` via age. Destructure to use one or the other separately.
- **`encrypt(data, recipients, opts?)`** — Seal `data` to one or more recipients. Input accepts `string` / `Uint8Array` / `ArrayBuffer`; `recipients` accepts a single string or an array. Default output is the binary age format as `Uint8Array`. Passing `opts.armored: true` wraps it in age's ASCII armor — the `-----BEGIN AGE ENCRYPTED FILE-----` banner + base64 body that's safe to embed in JSON / YAML / email. Either form decrypts via the same `decrypt` call (auto-detected by the leading bytes). Multi-recipient encryption uses age's native multi-recipient header — any one of the listed identities can decrypt the resulting ciphertext, no re-encryption per reader needed.
- **`decrypt(ciphertext, identities)`** — Open an age-encrypted payload with one of the supplied identities. **Auto-detects binary vs armored input** by sniffing the leading bytes for age's armor banner (whitespace and BOM bytes ahead of the banner are tolerated, so ciphertext pasted from JSON / email containers still detects correctly). age then walks the identities and uses the first that matches a stanza in the header. Returns `Uint8Array`; let scripts decode to a string via `api.text.charset.decode(bytes, "utf-8")` if appropriate (goja doesn't ship `TextDecoder`).
- **`rekey(ciphertext, oldIdentities, newRecipients, opts?)`** — Re-encrypt a payload for a fresh recipient set without ever exposing the plaintext to the caller. The decrypt+encrypt loop is internal; the plaintext lives only in this function's stack until the encrypt step finishes. Output format defaults to **match the input** — armored in / armored out, binary in / binary out — which is what you almost always want when key-rotating a payload that lives in a fixed location (file, vault row, JSON field). `opts.armored: true | false` forces the output regardless of input. Internally consistent with `encrypt` and

`decrypt` : same cross-checks (private as recipient, public as identity), same auto-detect on the input side, same multi-recipient handling on the output side.
- **`detectBackend(input)`** — Classify a recipient or identity string by encryption backend without parsing or I/O. Returns `{ backend: "age" | "pgp" | "unknown", kind?: "public" | "private" }`. Age recognises three input forms: bech32 X25519 (`age1...` , classified `kind: "public"`), `AGE-SECRET-KEY-1...` (`kind: "private"`), and SSH public keys (`ssh-rsa` / `ssh-ed25519` , classified `kind: "public"` since age accepts them as recipients). PGP recognises armored block markers (`-----BEGIN PGP PUBLIC KEY BLOCK-----` / `-----BEGIN PGP PRIVATE KEY BLOCK-----`). Anything else returns `{ backend: "unknown" }` — false-negative on unfamiliar input is the safer default than guessing. v0.5.8 ships the classifier only; sercon's actual `encrypt` / `decrypt` paths still handle age recipients only. Useful for scripts that need to dispatch on the format ("is this an age recipient or a PGP block?") before deciding which encrypt path — e.g., a config loader that routes some recipients through `api.crypto.encrypt.encrypt` and shells out to `gpg --encrypt` for others.

Cross-checks fire at the binding boundary so the common JS-side mistakes throw with named-key hints rather than cryptic bech32 errors deep inside age:

- **Private key as recipient** ("looks like a private key (AGE-SECRET-KEY-...); recipients are public keys (age1...)") — catches a script that mixed up the two halves on the encrypt call.
- **Public key as identity** ("looks like a public key (age1...); identities are private keys (AGE-SECRET-KEY-1...)") — inverse guard on the decrypt call.

Decryption with an identity that wasn't on the recipient list throws with age's own wording forwarded through: `"identity did not match any of the recipients: incorrect identity for recipient block"` . Scripts that want to silently try multiple identities should pass them all in one call (age tries each in turn internally) rather than catching this error.

SQLite (`api.db.sqlite.*`) is sercon's first **stateful-handle** binding — the shape future protocol bindings (redis / ldap / …) will reuse. The namespace exposes only `open` ; the object it resolves to carries the real surface. The driver is [`modernc.org/sqlite`](modernc.org/sqlite) , the pure-Go SQLite translation — no cgo, so the project's cgo-free rule is preserved.

- **`open(path)`** — Connect to a database. `":memory:"` is an in-RAM database that evaporates when the handle is closed; any other string is a filesystem path (created if absent). The connection is `Ping` -ed before `open` resolves, so a bad path surfaces here rather than at the first query. Resolves to a handle object: `{ exec, query, queryValue, close }` .
- **`exec(sql, ...params)`** — Run a non-query statement (DDL, INSERT, UPDATE, DELETE). Returns `{ rowsAffected, lastInsertId }` — both are driver-reported and may be zero for statements that don't populate them (e.g. `CREATE TABLE`).
- **`query(sql, ...params)`** — Run a SELECT and return one object per row, keyed by column name. Column types map as: INTEGER → number, REAL → number, TEXT → string, NULL → `null` , BLOB → `Uint8Array` (unless the bytes are valid UTF-8, in which

case they promote to a string — SQLite stores TEXT and BLOB the same way at the storage layer, so this heuristic recovers the common TEXT case while keeping genuinely-binary BLOBs as bytes).

- **`queryValue(sql, ...params)`** — Run a statement expected to produce a scalar and return the first column of the first row, or `null` when no rows match. Rows beyond the first are discarded. Ideal for `SELECT count(*)`, `PRAGMA user_version`, single-column lookups.
- **`begin()`** — Open a transaction; resolves to a nested handle with the same `exec` / `query` / `queryValue` surface plus `commit()` and `rollback()`. The transaction reuses the exact same code paths as the top-level handle (via an internal executor interface that both `*sql.DB` and `*sql.Tx` satisfy) — only the error prefix differs (`sqlite.tx.exec:` vs `sqlite.exec:`). A transaction holds a pooled connection until it commits or rolls back, so **every `begin()` must reach one of them** — there's no auto-rollback on handle close in this cut. Once finalized, the transaction is spent: further calls throw `sql: transaction has already been committed or rolled back`, and so do double-commit / commit-after-rollback. A constraint violation inside the transaction throws (with the `sqlite.tx.exec:` prefix) but doesn't auto-roll-back — the script decides whether to retry or roll back.
- **`prepare(sql)`** — Compile a statement once and resolve to a handle whose `exec` / `query` / `queryValue` run it repeatedly with fresh bind params. Unlike the handle / transaction methods, these take **only the params** — no SQL string — since the SQL was fixed at `prepare()` time. Worthwhile for batch-insert or repeated-lookup loops where the per-call parse + plan cost would otherwise dominate. Invalid SQL throws at `prepare()`, not the first `exec()`. Like every handle here it must be `close()`d; a leaked statement pins driver resources. Statements are bound to the database handle, not a transaction — using a prepared statement inside a transaction is out of scope for now.
- **`close()`** — Release the connection. There's no finalizer — scripts that forget to close leak a connection until the process exits. The documented pattern is open / use / close.

Parameters bind positionally as `?` placeholders, in argument order. goja exports JS numbers as int64 / float64, strings as string, `null` as nil, and `Uint8Array` as `[]byte` — all of which the driver accepts directly, so no per-type coercion is needed on the script side. Every handle and transaction method is async (Promise-returning); `open` and `begin` are too.

Hash bindings interpret the input as a UTF-8 byte sequence and return lowercase hex. SHA-3 functions are `sha3_256` / `sha3_512` (the IETF spec uses the underscore so the JS name matches recon's). BLAKE3 uses the upstream `lukechampine.com/blake3` reference implementation with a 32-byte output. `crc32` is the IEEE polynomial, zero-padded to 8 hex chars.

String utilities (`api.text.str.*`) follow PHP-/recon-style semantics where they differ from JS:

- `trim` / `ltrim` / `rtrim` accept an optional mask string; **any character in the mask** is stripped. Default mask is whitespace (`" \t\n\r\v\f"`).

- `reverse` is rune-aware, so `reverse("café")` is `"éfac"`. Grapheme clusters are not handled — that's a separate problem.
- `nl2br` emits `<br>` by default; pass `true` as the second arg for XHTML-style `<br/>`.
- `urlEncode` / `urlDecode` use the form encoding (`+` for space). For path-segment encoding, use `encodeURIComponent` instead — that one is provided by goja natively.
- `pad` / `lpad` / `rpad` follow recon's `str_pad`: `pad` defaults to side `"right"`; `"left"` and `"both"` are also accepted.
- `sprintf` / `printf` are thin wrappers over Go's `fmt`, so the verb set is Go's (`%s`, `%d`, `%x`, `%.2f`, `%v`, `%t`, `%q`, etc.) — not PHP's. `printf` writes to stdout.
- `normalizeNewlines` canonicalises any mix of `\r\n`, `\r`, and `\n` to the requested style.

Path utilities (`api.fs.path.*`) are POSIX (forward-slash). On Windows either pass forward-slash paths or convert separators yourself.

Time formatting (`api.runtime.time.format(unixMs, fmt, tz?)`) takes Unix milliseconds (symmetric with `api.runtime.time.nowMs()`) and a small strftime token set: `%Y %y %m %d %H %M %S %F %T %j %A %a %B %b %z %Z %%`. Day and month names are in English. Pass a third-argument IANA zone name (e.g. `"UTC"`, `"America/New_York"`) to render in that zone; if omitted, the host's `time.Local` is used. Unknown `%X` tokens are emitted verbatim so a typo is visible rather than silently dropped.

Protocol probes (`api.net.*`) are stdlib-backed and hit the real network. All three return Promises (via `PromisifyAsync`) and accept an optional `{ timeout: <ms> }` second arg; the default is 5 seconds.

- **`tcp(target, opts?)`** — dials TCP, measures round-trip from resolution to handshake. `target` is `"host:port"` or just `"host"` (with `opts.port` overriding the default `80`). Resolves to the remote IP, port, and `latencyMs`.
- **`dns(host, opts?)`** — looks up A, AAAA, MX, TXT, CNAME, and NS records via `net.Resolver`. Pass `opts.types` to scope the query; empty record sets are omitted from the result object so scripts can probe membership with `if ("mx" in result)`.
- **`tls(target, opts?)`** — connects with `InsecureSkipVerify` so even expired or hostname-mismatched certs come back inspectable. Returns the leaf cert's CN, issuer CN, validity window, `daysRemaining`, `dnsNames`, serial number, and a SHA-256 fingerprint (lowercase hex). Default port is `443`. Hosts that care about validity should re-validate the cert themselves.
- **`ntp(host, opts?)`** — queries an NTPv4 server (UDP 123 by default) via `github.com/beevik/ntp`. Returns the server time (ISO 8601), local-clock offset, round-trip time, stratum, reference time, and the root delay / dispersion values from the server's response. All durations are in milliseconds with sub-millisecond precision.
- **`whois(domain, opts?)`** — two-hop WHOIS lookup (IANA → registrar's whois server) via `github.com/likexian/whois` with parsing through `github.com/likexian/whois-parser`. `raw` is always populated; `domain` and `registrar` are best-effort parsed views (some TLDs the parser doesn't understand will return only `raw`). The whois library

doesn't accept a `context.Context` — `opts.timeout` is plumbed through its own per-client setting and the engine's `Options.Timeout` watchdog catches the outer call.

- **`ping(host, opts?)`** — Reachability probe in two modes. `"tcp"` (default) dials `host:port` `count` times and measures connect RTT — no privileges, works in containers / CI, and "can I open a connection" is what most liveness checks actually want. `"icmp"` does real ICMP echo via `pro-bing` but needs raw-socket privileges (root / CAP_NET_RAW), so it's opt-in. Returns `{ host, ip, mode, sent, received, lossPercent, minMs, avgMs, maxMs }`. An unreachable host resolves with `received: 0` / `lossPercent: 100` — "down" is a normal outcome, not a throw; only DNS-resolution failure and bad arguments throw.
- **`smtp(host, opts?)`** — SMTP capability probe (not a send pipeline). Opens the connection, reads the banner, sends EHLO, and reports the advertised extensions: `starttls` (boolean), `authMechanisms` (e.g. `["PLAIN", "LOGIN"]`), `sizeLimit`, and the raw `extensions` list. Hand-rolled over `net/textproto` (net/smtp doesn't expose the banner or full extension list as data). No mail is sent — it QUITs after EHLO. Connection / protocol failures throw; a server that doesn't advertise STARTTLS / AUTH reports them as `false` / `[]` (a finding, not an error).
- **`wss(url, opts?)`** — WebSocket handshake probe via `coder/websocket`. Opens the `ws://` / `wss://` connection, optionally measures a ping/pong RTT (`opts.ping`, default true), and closes. Returns `{ url, connected, subprotocol, status, handshakeMs, pingMs }` — `status` is the 101 upgrade response code, `pingMs` is `-1` when the ping was skipped or the server didn't pong. A failed handshake (non-101, refused, bad URL) throws; there's no useful partial result for a failed upgrade. Not a streaming client — the connection doesn't outlive the call.

Email-authentication probes (`api.net.email.*`) read DNS records and surface the published policy. They all return `{ present: false }` when the relevant record is absent (NXDOMAIN or no TXT record matching the marker prefix), so scripts can write a single presence-check pattern across the family.

- **`spf(domain)`** — queries `TXT(<domain>)` and returns the first record starting with `v=spf1`. `mechanisms` is the tokenised sequence after the version prefix; `allPolicy` summarises the trailing `all` qualifier (`"pass"` for `all` / `+all`, `"fail"` for `-all`, `"softfail"` for `~all`, `"neutral"` for `?all`, empty string when no `all` is present).

- **`dmarc(domain)`** — queries `TXT(_dmarc.<domain>)` and parses the `v=DMARC1; key=val; …` form into a flat tag map (keys are case-folded; values retain internal whitespace and commas). `policy` / `subdomain` / `percent` / `rua` / `ruf` are surfaced separately because those are the fields most scripts care about.

- **`mtaSts(domain)`** — looks up `TXT(_mta-sts.<domain>)` and, on success, fetches the policy file at `https://mta-sts.<domain>/.well-known/mta-sts.txt`. The TXT carries a versioned `id` for change detection; the policy file is where the actual `mode` + `mx` list live. RFC 8461 format (line-based, `key: value`, `mx:` repeats) is parsed inline. Policy-fetch failures (TLS errors, 4xx, timeout) don't fail the binding — the TXT part is still returned, and the fetch error surfaces under `policyError` so scripts can decide what to do.

- **`tlsRpt(domain)`** — looks up `TXT(_smtp._tls.<domain>)` and parses the `v=TLSRPTv1; rua=…` form into a tag map. `rua` is surfaced separately because that's the actionable bit.
- **`bimi(domain, opts?)`** — looks up `<selector>._bimi.<domain>`, selector defaulting to `default`. Surfaces the logo URL (`l`) and VMC URL (`a`) from the tag map.
- **`all(domain)`** — runs every probe above in parallel via goroutines and returns a single aggregate object keyed by probe name. Per-probe failures don't fail the aggregate — they surface under `<probe>.error` so a partial result is still useful (e.g. SPF + DMARC found, MTA-STS policy fetch timed out).

`api.ui.tui.*` — multi-pane TUI

Scripts that orchestrate multiple subprocesses (e.g. `brew`, `npm`, `cargo` updates running in parallel) can declare a multi-pane terminal layout and route each subprocess's stdout/stderr into its own pane. Activation is **script-driven**: the first call to `api.ui.tui.layout(...)` enters TUI mode. If stdout is not a TTY (CI, pipes, `make demo`), the same calls fall back to prefixed plain-text lines (`[paneName] line`) so the same script runs in both contexts.

Layout

```
api.ui.tui.layout({
  rows: [
    { name: "log", title: "Orchestrator", weight: 1 },
    { cols: [
        { name: "brew" },
        { name: "npm" },
      ],
      weight: 2,
    },
  ],
});
```

Each layout node is one of:

- `{ name: string, title?: string, weight?: number }` — a leaf pane.
- `{ rows: LayoutNode[], weight?: number }` — a vertical split.
- `{ cols: LayoutNode[], weight?: number }` — a horizontal split.

`weight` defaults to 1. Pane names must be unique across the whole tree. `layout` may be called **once** per Run; a second call throws.

Pane handles

```
const log = api.ui.tui.pane("log");
log.writeln("Updating Homebrew…");
log.title("Done");
log.clear();
log.write("partial line ");
```

`api.ui.tui.pane(name)` returns a handle with `write`, `writeln`, `clear`, `title`. Methods are synchronous from the script's perspective — they enqueue to the TUI goroutine.

Subprocess routing

`api.tools.exec.shell(cmd, opts)` accepts `opts.pane` (a Pane handle or a pane name string):

```
await api.tools.exec.shell("brew update && brew upgrade", { pane: "brew"
});
```

When set, the subprocess's stdout **and** stderr stream into the pane line by line. The returned `{ exitCode, durationMs, success }` is the same as without `pane:` except `stdout` and `stderr` are empty (data was streamed, not captured). ANSI colors are translated to tview color tags and rendered natively; `\r` without `\n` overwrites the current line so progress spinners render cleanly.

Keybindings (TTY mode only)

Key	Action
`Tab` / `Shift-Tab`	Cycle focus through panes
`PgUp` / `PgDn`	Scroll focused pane one page
`↑` / `↓`	Scroll focused pane one line
`Home` / `End`	Jump to top / bottom
`Ctrl-C`	Abort the script

The focused pane has a yellow border; the bottom status bar shows its name and the active keys.

Limitations (v1)

- `api.ui.tui` is **incompatible with `--watch`**: calling `api.ui.tui.layout()` under `--watch` throws. Use one or the other.
- No mouse, no runtime layout mutation, no input panes (`api.ui.tui.input`), no snapshot-to-normal-screen on exit. The alt screen is restored at script end and the usual `PASS` / `FAIL` line prints.

6. JavaScript runtime built-ins (goja)

`scriptengine` runs on goja, which implements **ES5.1 + a large subset of ES6+**. The following are present and behave per spec:

Globals

- `Object`, `Array`, `String`, `Number`, `Boolean`, `BigInt`
- `Math`, `Date`, `JSON`, `RegExp`
- `Error`, `TypeError`, `RangeError`, `SyntaxError`, `ReferenceError`, `EvalError`, `URIError`
- `Promise` (provided via goja's native implementation; resolution microtasks are pumped by the event loop)
- `Symbol` (partial: well-known symbols `Symbol.iterator`, `Symbol.asyncIterator`, etc., are supported)
- `Proxy`, `Reflect` (partial)

```
api.runtime.log(Math.PI.toFixed(4), Math.max(1, 9, 3));
api.runtime.log(new Date().toISOString());
api.runtime.log(JSON.stringify({ a: 1, b: [2, 3] }));
api.runtime.log("abc".repeat(3), "abc".padStart(6, "_"));
```

Collections

- `Map`, `Set`, `WeakMap`, `WeakSet`

```
const counts = new Map<string, number>();
for (const ch of "banana") counts.set(ch, (counts.get(ch) ?? 0) + 1);
api.runtime.log([...counts]); // [["b",1],["a",3],["n",2]]
```

Typed arrays

- `ArrayBuffer`, `DataView`
- `Int8Array`, `Uint8Array`, `Uint8ClampedArray`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `Float32Array`, `Float64Array`

```
const buf = new ArrayBuffer(4);
new DataView(buf).setUint32(0, 0xCAFEBABE);
api.runtime.log(new Uint8Array(buf)[0].toString(16)); // ca
```

Iteration and generators

- `for…of`, `for…in`, spread, destructuring

- Generator functions (`function*`)
- Async generators (`async function*`) — supported via goja's event loop

```
function* fib() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}
const it = fib();
const first10 = Array.from({ length: 10 }, () => it.next().value);
api.runtime.log(first10);
```

Not (yet) provided

- `fetch` — use `api.net.http.*` from the CLI, or register your own binding.
- `Worker` , threading primitives.
- DOM globals (`window` , `document`).
- Native ES modules (`import` / `export` are *transpiled* to CommonJS at load time — see section 8).

7. Async runtime additions (goja_nodejs)

The event loop bundles a small Node-compatible runtime on top of goja:

Timers

```
setTimeout(() => api.runtime.log("after 50ms"), 50);
setInterval(() => api.runtime.log("tick"), 100);
setImmediate(() => api.runtime.log("next tick"));
clearTimeout(t); clearInterval(i); clearImmediate(im);
```

The event loop holds the script alive while *any* timer is pending. If your script ends with only `setTimeout` callbacks scheduled, the run will wait for them to fire before returning.

`console`

```
console.log("info");
console.error("oops");
console.warn("careful");
console.info("fyi");
console.debug("trace");
```

Disable with `Options.DisableConsole = true` if you want a clean global namespace.

`require`

CommonJS require, with TS-aware extension fallback added by sercon (see section 10):

```
const helpers = require("./helpers");
helpers.doThing();
```

Both `require("./foo")` and `import { x } from "./foo"` resolve to the same module instance per Run; esbuild rewrites `import` to `require` at transpile time.

8. TypeScript support

TS is transpiled by esbuild in-process. There is no `tsc`, no `tsconfig.json`, no type-checking — types are erased and the resulting JS is what executes. Practical implications:

- All TS syntax esbuild supports is allowed: type annotations, `interface`, `type`, generics, enums (compiled to objects), namespaces, decorators (legacy & TC39 stage 3).
- `import type { X } from "y"` is stripped entirely (no runtime require).
- TS errors are *not* surfaced as build errors — only esbuild parse errors are. Run a separate `tsc --noEmit` in CI if you want type enforcement.
- `tsconfig.json` is **not** consulted.

`.tsx` and `.jsx` are supported via esbuild's `LoaderTSX` / `LoaderJSX`, including for required modules resolved by the engine's extension fallback. JSX has no React runtime in scope by default — either set the factory at the file level with an `@jsx` pragma:

```
/** @jsx h */
function h(tag: string, props: any, ...children: any[]) {
  return { tag, props: props ?? {}, children };
}
export const Box = (label: string) => <div className="box">{label}</div>;
```

…or provide a `React.createElement`-compatible binding from Go and rely on esbuild's default pragma. ESM `export default <value>` also works through the entry-script rewriter's `__esModule ? .default : m` unwrap, so `import answer from "./mod"` resolves to the default export.

9. Top-level `await`

Top-level `await` works in entry scripts:

```
const r = await api.net.http.get("https://example.com");
api.runtime.log(r.status);
```

How it works under the hood: esbuild rejects top-level await with the `cjs` output format, so the engine transpiles the *entry* script with `format=esm`, then rewrites `import` statements to `require()` calls and wraps the rest of the body in:

```
;(async () => { /* your transpiled body */ })().then(__resolve, __reject);
```

`__resolve` and `__reject` are bindings the engine sets on the runtime to capture the final settlement. *Required-module* code (anything loaded through `require`) is transpiled with `format=cjs` and does **not** support top-level await — wrap in `(async () => …)()` yourself if you need it inside a module.

10. Module resolution

The engine plugs a custom source loader into `goja_nodejs/require` that adds:

1. Direct path: if the requested file exists on disk, use it. `.ts` / `.tsx` files are transpiled on the fly.
2. **Extension fallback** when the request has no extension: try `.ts`, `.tsx`, `.js`, `.cjs`, `.mjs`, `.json` in that order.
3. **`.js` → `.ts` swap**: if a path ending in `.js` / `.cjs` / `.mjs` is requested but the literal file doesn't exist, the same path ending in `.ts` (or `.tsx`) is tried. Handles `package.json` `main` fields that point at compiled output where only the TypeScript source is on disk.
4. **`package.json` `source` preference**: when reading a `package.json`, if a `source` field is present and points at an existing `.ts`/`.tsx` file, `main` is rewritten to that path before the registry consumes it. Matches the convention used by parcel, microbundle, and similar bundlers to mark the TS source of truth.
5. **Directory index**: if the resolved path is a directory, try `index.ts`, `index.tsx`, `index.js`.

The base directory follows Node's CommonJS rules — relative imports resolve against the requiring module's directory, courtesy of goja_nodejs.

```
// All resolve to ./helpers/assert.ts:
import { check } from "./helpers/assert";
import { check } from "./helpers/assert.ts";
const { check } = require("./helpers/assert");
const { check } = require("./helpers/assert.js");
```

JSON imports work out of the box via either the ESM-style default import (esbuild rewrites it to a require) or a direct `require`:

```
import data from "./data.json";
const r = require("./data.json");
```

`node_modules` lookup *works* via the upstream registry, but the source loader doesn't currently transpile through that path — keep TS helpers under `ScriptRoot`.

11. Timeouts and cancellation

Two mechanisms interrupt a running script:

- `Options.Timeout` — wall-clock limit per `Run`. Exceeding it returns `scriptengine.ErrScriptTimeout`.
- `ctx` passed to `Run` / `RunFile` — cancelling the context returns `ctx.Err()` (typically `context.Canceled` or `context.DeadlineExceeded`).

Both call `vm.Interrupt(...)` and `loop.Terminate()` from a watcher goroutine. This works for **sync** JS — including `while(true){}` — because the goja VM checks the interrupt flag between bytecode steps. A host Go binding that's blocked in cgo or a long syscall isn't interruptible mid-call; if you write such a binding, plumb the engine `ctx` through to it yourself.

```
eng := scriptengine.New(scriptengine.Options{Timeout: 200 *
time.Millisecond})
_, err := eng.Run(ctx, "loop.ts", `while (true) {}`)
// err is scriptengine.ErrScriptTimeout, returned ~200-300ms after Run.
```

12. Error semantics

- A Go binding returning `(T, error)` automatically throws as a JS exception when `err != nil`. The exception's `.message` is `err.Error()`.

  ```
  try { mayFail(); } catch (e) { api.runtime.log(String(e)); }
  ```

- A Go binding that `panic`s with a `*goja.Object` (e.g. `vm.NewGoError(...)`) does the same.

- A script throwing a JS error out of the top level surfaces from `Run` as a Go `error` whose message is the JS error's `.message`.

- Timeout errors satisfy `errors.Is(err, scriptengine.ErrScriptTimeout)`. Cancellation errors satisfy `errors.Is(err, context.Canceled)` or `context.DeadlineExceeded`.

13. Type generation (.d.ts)

`Engine.WriteTypes(w)` walks the registered bindings and emits a TypeScript declaration file. The mapping table (abbreviated):

Go type	TS type
`string`	`string`
any numeric / `float64`	`number`
`bool`	`boolean`
`[]T`	`T[]`; `[]byte` → `Uint8Array`
`map[string]T`	`Record<string, T>`
`*T`	`T` (transparent unwrap)
`error` (single return)	omitted (collapses to `void`)
`(T, error)` (tuple return)	`T`
`interface{}` / `any`	`unknown`
`goja.FunctionCall` parameter	folded into `(...args: unknown[])`
`goja.Value` return	`unknown`
`scriptengine.AsyncBinding` value (from `PromisifyAsync[T]`)	`Promise<T>`
self-referential types	`unknown` (cycle detection bails after depth 4)

```
sercon --emit-dts api.d.ts
```

In your editor, place the resulting `.d.ts` next to your scripts (or reference it via `/// <reference path="..." />`) for autocomplete.

JSDoc comments on declarations

The emitter pulls JSDoc strings from the engine's doc map (set via `Engine.SetDocs` / `Engine.SetMemberDocs`) and renders them above each declaration. Single-line docs collapse to `/** … */`; multi-line docs expand to a standard `* `-prefixed block. Bindings without a doc entry emit no JSDoc — the emitter never inserts empty placeholders. Docs are looked up by the same dotted path the binding was registered under, so namespace members get hover text too:

```
// AUTO-GENERATED by scriptengine.Engine.WriteTypes — do not edit by hand.

/** sercon's built-in script surface. … */
declare const api: {
  hash: {
    /** SHA-256 hex digest of a UTF-8 input. */
    sha256(...args: unknown[]): string;
    /** BLAKE3 hex digest (32-byte output, lukechampine.com/blake3). */
    blake3(...args: unknown[]): string;
    // …
  };
  // …
};
```

The CLI's `api.*` surface ships with a curated doc map; see `cmd/sercon/api_docs.go` for the source of truth and add new entries there when adding new bindings (the lockstep rule keeps `--examples` / MANUAL / `api.d.ts` in sync, and the doc map is now the seventh artifact in that chain).

14. Limitations and gotchas

- **No native ES modules at runtime.** All `import` is rewritten to `require` by esbuild. Side-effect-only imports run the module body exactly once per Run.
- **No top-level await inside required modules.** Wrap with `(async () => … )()` inside the module if you need it.
- **No `tsconfig.json` honoured.** Module resolution and type checking are independent from any project-level TS config.
- **Per-Run runtime.** Side effects on `globalThis` do not persist between calls to `Run` on the same `Engine`. The require *compile* cache is shared; module *exports* are not.
- **Multi-line ESM imports may stress the entry rewriter.** The current rewriter is line-based and handles the common forms; especially gnarly inputs (comments inside the spec list, very unusual whitespace) fall back to executing as-is, which may throw.
- **`RegisterConstructor` is d.ts-only today.** At runtime it behaves like `Register`. True `new`-able constructor semantics are on the roadmap.
- **HTTP bindings are real network calls.** `api.net.http.*` uses `net/http` with a 5s timeout. They are not mockable from JS.

See OUT-OF-SCOPE.md for the active backlog of deferred ideas.

This manual covers sercon v0.7.0. Whenever you add, remove, or change a flag, a binding, or the script API, update this file alongside the help screen (`--help`), the examples walkthrough (`--examples`), and the `CHANGELOG.md`.